Adaptive Low-Rank Memory Compression Framework for Operating Systems

Kirollos Kamel
Cairo University

Eissa Hozayen
Cairo University

Fady Fawzy
Cairo University

Fady Ashraf
Cairo University

Kirollos Sameh
Cairo University

Omar Youssef
Cairo University

Jody Ahmed
Cairo University

Ereny Emad
Cairo University

Fady N. Eleshary
Cairo University

Habiba Ahmed
Cairo University

Mentored by:
Dr. Samah ElTantawy
Cairo University

***Abstract*—This paper presents a novel memory compression framework integrated into the operating system memory manager, leveraging low-rank matrix approximation via Singular Value Decomposition (SVD) to reduce RAM consumption. Instead of storing memory pages in their raw form, the system dynamically analyzes whether the data within a page exhibits low-rank structure. When such structure exists, the page is compressed using truncated SVD, retaining only the most significant singular components. The framework introduces an adaptive rank selection mechanism that dynamically adjusts compression aggressiveness based on real-time memory pressure, and a dual-threshold decision model that evaluates both compression ratio and reconstruction error before applying compression. Experimental validation across five data categories demonstrates that the system achieves up to 87.5% memory reduction for structured data while reliably rejecting non-compressible random data. Results confirm that compressibility is fundamentally determined by the underlying structure of the data rather than its size, establishing a data-aware approach to memory management that balances efficiency with accuracy under varying system constraints.**

***Index Terms*—Memory Compression, Low-Rank Approximation, Singular Value Decomposition, Operating Systems, Virtual Memory, Adaptive Compression.**

Manuscript prepared April 2026.

I. Problem Definition

The global RAM (memory) shortage is a structural supply-demand imbalance in semiconductor memory, especially DRAM and VRAM. The main cause of this crisis is the massive growth in AI and data center memory demand. Combined with limited manufacturing capacity, this has created a significant imbalance in the supply and demand of RAM.

This crisis has led to a sharp increase in RAM prices, which have risen by as much as 171% year-over-year, with some DDR5 prices quadrupling since late 2025 [13]. Massive AI data centers are projected to consume over 70% of high-end DRAM (DDR5) in 2026 [14]. Furthermore, major companies like NVIDIA and Samsung have shifted production from traditional DRAM to HBM (High-Bandwidth Memory), which is mainly used in AI accelerators. However, HBM is more complex to manufacture and has lower yields, contributing massively to the current crisis [15].

A. Approaches to Addressing the Shortage

The RAM shortage can be addressed through multiple approaches:

1) **Increase Manufacturing Capacity:** A long-term approach focusing on building new facilities for RAM production, which will take years to show fruition.
2) **Rebalance Production Focus:** Entirely in the hands of major memory production companies to shift some production capacity back to traditional DRAM production.
3) **Software and System Optimization:** Focuses on improving memory efficiency in operating systems and applications by employing various compression and optimization techniques.

This research focuses on the third approach, investigating techniques for data compression using Linear Algebra, with the primary goal of reducing overall memory consumption. By optimizing how data is stored and managed in RAM, this approach aims to help alleviate and stabilize the current memory shortage.

II. Literature Review

A. Unified Memory Access Performance

Landaverde et al. [6] investigated the performance of Unified Memory Access (UMA) in CUDA as a method to simplify memory management between the CPU and GPU. Their study analyzed how UMA handles memory transfers when working with data structures represented as matrices, which are common in linear algebra and scientific computing. The results showed that although UMA simplifies programming by allowing the CPU and GPU to share a unified memory space, it often introduces performance overhead compared to traditional explicit memory transfers. However, its limitation lies in its reliance on explicit memory access patterns, which are highly application-specific and cannot adapt well to unstructured workloads without significant developer intervention.

B. Compressed Linear Algebra for Machine Learning

Compressed Linear Algebra (CLA) [7] is a technique designed to improve the performance of large-scale machine learning workloads that rely heavily on repeated matrix–vector operations and require datasets to fit in main memory. CLA applies lightweight database-style compression techniques to matrices and allows linear algebra operations to be performed directly on the compressed data without decompression. A key limitation of CLA is that it requires offline compression planning and explicit application-level integration, making it unsuitable for transparent, system-wide memory management.

C. Tucker Tensor Decomposition for Large-Scale Data

Large scientific simulations often generate datasets that reach several terabytes. Ballard et al. [8] proposed compressing such datasets by representing them as tensors and applying the Tucker Tensor Decomposition to approximate the data using a smaller structure composed of a core tensor and several factor matrices. Despite its massive compression capability, the primary limitation is its high computational overhead, rendering it impractical for real-time memory compression during active execution.

D. GPU Memory Oversubscription

Shao et al. [9] studied the performance limitations caused by the restricted memory capacity of GPUs, which can lead to memory oversubscription and severe performance degradation. The main cause of slowdown is inefficient memory access patterns: when applications request data randomly, the GPU must repeatedly evict and reload memory pages between GPU memory and system RAM, creating a phenomenon known as *thrashing*. The researchers evaluated several optimization hints, including `AccessedBy`, `ReadMostly`, and `PreferredLocation`, to mitigate this effect. A notable limitation of this approach is its reliance on these software hints, which places the burden of memory optimization directly on the programmer rather than the operating system.

E. Adaptive Page Migration

Ganguly et al. [10] addressed the problem of memory oversubscription in GPU systems. Instead of moving data randomly, the proposed system uses hardware access counters to check the "importance" of the data first. Frequently used data is moved to GPU RAM for better speed, while rarely used data is accessed via zero-copy without migration. While effective, this method is limited by its dependence on specific hardware access counters, reducing its portability across different architectures, and it fails to physically compress data to save absolute memory.

F. Matrix Compression via Randomized Low-Rank Factorization

Saha et al. [11] proposed a method that represents a matrix A as the product of two smaller matrices L and R, significantly reducing storage requirements. The method also applies low-precision quantization to further compress the data. The algorithm estimates a basis for the range space of the matrix using randomized sampling of its columns, then quantizes this basis and computes approximate projections. However, its reliance on precision quantization introduces irreversible data loss, which restricts its applicability strictly to error-tolerant machine learning tasks rather than general-purpose system memory.

G. FlashSVD: Memory-Efficient Inference

FlashSVD [12] addresses the Activation Memory Paradox, which explains why compressing an AI model's weights using SVD does not necessarily reduce overall memory usage during inference. To overcome this, FlashSVD introduces Streaming Kernels: small sub-blocks of input data and compressed weights are loaded directly into the processor's on-chip SRAM (L1 cache), where partial computations are performed locally. Its major limitation is its exclusive design for neural network inference (activation memory), meaning it cannot generalize to arbitrary system workloads or dynamic memory pages.

H. Research Gap and Contribution

Most existing approaches suffer from a common set of limitations: they either require explicit application-level integration (CLA, UMA, GPU Oversubscription hints), introduce high computational overhead (Tucker Tensor), depend on specialized hardware (Adaptive Page Migration), or target highly specific, error-tolerant workloads (FlashSVD, Randomized Factorization).

Building on previous research, this paper addresses these limitations by proposing a transparent, automatic compression mechanism at the operating system level. Our solution eliminates the need for developer intervention, generalizes across application types by treating raw memory pages as matrices, and uses an adaptive low-overhead decision model to ensure real-time feasibility without relying on specific hardware.

III. System Overview

The proposed method introduces a memory compression framework integrated into the operating system memory manager. Instead of storing memory pages in their raw form, the system dynamically analyzes whether the data within a page exhibits low-rank structure. When such structure exists, the page is compressed using matrix factorization techniques derived from Singular Value Decomposition (SVD) [1], [2].

Each memory page is treated as a matrix representation of the stored data, enabling the operating system to apply mathematical compression techniques that preserve most of the information while reducing memory usage.

The system operates in three main stages:

1) Page structure detection
2) Low-rank compression
3) Adaptive rank adjustment during memory pressure

IV. Mathematical Model

A. Mathematical Representation of Memory Pages

Let a memory page be represented as a matrix:

$$A \in \mathbb{R}^{m \times n} \tag{1}$$

where m is the number of rows, n is the number of columns, and each element represents stored data.

For example, a 4KB memory page may be reshaped into matrices such as 64×64 or 128×32. This transformation does not modify the data but enables matrix operations to be applied.

B. Low-Rank Approximation Model

Many matrices generated by applications contain redundancy or correlations [5], meaning they can be approximated using fewer components. The matrix A can be approximated as:

$$A \approx LR \tag{2}$$

where $L \in \mathbb{R}^{m \times k}$, $R \in \mathbb{R}^{k \times n}$, and $k \ll \min(m, n)$.

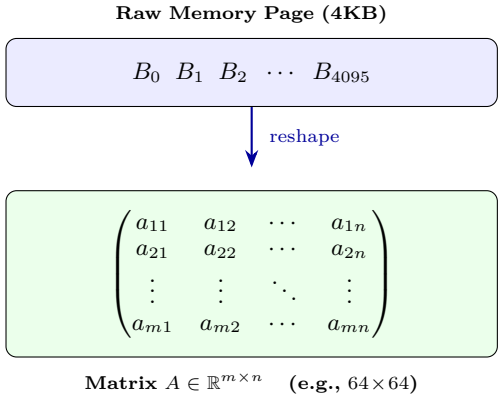


Fig. 1: Reshaping a raw memory page into a matrix representation for SVD-based compression.

The storage requirement becomes $mk + kn$ instead of mn, which significantly reduces memory usage when k is small.

C. Singular Value Decomposition Formulation

The optimal low-rank approximation is obtained using Singular Value Decomposition [1]:

$$A = U \Sigma V^T \tag{3}$$

where $U \in \mathbb{R}^{m \times m}$, $\Sigma \in \mathbb{R}^{m \times n}$, and $V \in \mathbb{R}^{n \times n}$.

To compress the matrix, only the largest k singular values are retained:

$$A_k = U_k \Sigma_k V_k^T \tag{4}$$

where $U_k \in \mathbb{R}^{m \times k}$, $\Sigma_k \in \mathbb{R}^{k \times k}$, and $V_k \in \mathbb{R}^{n \times k}$. This truncated representation preserves the most significant information in the matrix.

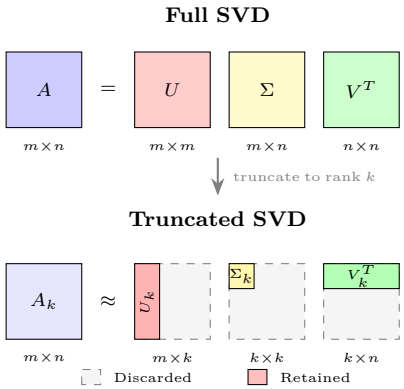


Fig. 2: SVD truncation: only the top k components (colored) are retained, reducing storage from mn to $mk + kn$ elements.

D. Memory Compression Ratio

The compression ratio is defined as:

$$\text{CR} = \frac{\text{Original Size}}{\text{Compressed Size}} = \frac{mn}{mk + kn} \tag{5}$$

Original storage requires mn elements; compressed storage requires $mk + kn$ elements.

1) Example: Let $m = 64$, $n = 64$, $k = 8$. Original elements: 4096. Compressed elements: $64 \times 8 + 8 \times 64 = 1024$. Compression ratio: $4096/1024 = 4$. Thus, the page uses approximately 75% less memory.

E. Approximation Error Model

Compression introduces approximation error. The reconstruction error is measured using the Frobenius norm [4]:

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2} \tag{6}$$

where σ_i are the singular values of A and A_k is the truncated approximation. This allows the system to control the trade-off between accuracy and memory usage.

F. Page Compression Decision Model

Not all pages benefit from compression. Therefore, the operating system performs structure detection. For each page matrix A:

1) Compute approximate rank using randomized sampling [3]
2) Estimate compression ratio
3) Evaluate approximation error

Compression is applied only if:

$$\text{CR} > T_c \tag{7}$$

$$\|A - A_k\|_F < \epsilon \tag{8}$$

where T_c is a compression threshold and ϵ is an acceptable error bound.

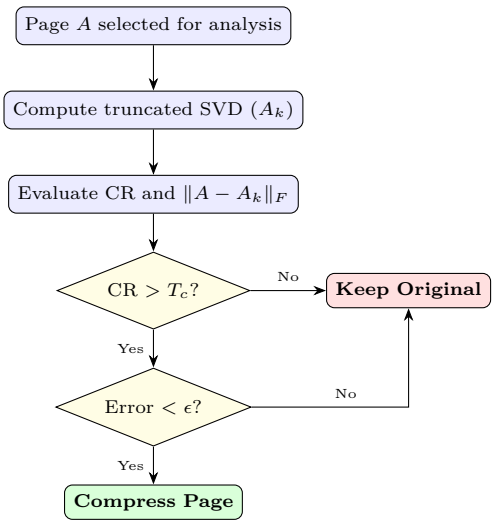


Fig. 3: Compression decision flowchart: a page is compressed only when both thresholds are satisfied.

G. Adaptive Rank Selection

The operating system dynamically adjusts the rank k based on system memory pressure. Let M_{avail} denote available memory and M_{req} denote required memory. Memory pressure is defined as:

$$P = \frac{M_{\text{req}}}{M_{\text{avail}}} \tag{9}$$

The rank selection rule is:

$$k = k_{\max}(1 - \alpha P) \tag{10}$$

where α is a scaling constant. Under low memory pressure, a higher rank is selected for higher accuracy. Under high memory pressure, a lower rank is selected for stronger compression.

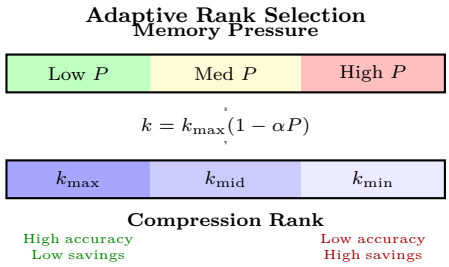


Fig. 4: Adaptive rank selection: increasing memory pressure reduces rank k, trading accuracy for stronger compression.

H. Memory Page Reconstruction

When a compressed page is accessed:

1) The page is reconstructed using $A_k = LR$
2) The reconstructed page is loaded into memory
3) After execution, the page may be recompressed

I. Integration with Virtual Memory System

The compression system integrates with the virtual memory subsystem. Each page maintains metadata including: compression flag, rank value k, matrix dimensions, and compression error estimate. Compressed pages are stored in a compressed memory pool managed by the operating system kernel.

V. Algorithm Workflow

The compression pipeline operates as detailed in Algorithm 1.

A. Computational Complexity

Full SVD has complexity $O(mn^2)$ [1]. To reduce computational overhead, randomized SVD is used with complexity $O(mnk)$, which is efficient for page-sized matrices.

Algorithm 1 Adaptive Page Compression Pipeline

Input: Memory page P, Memory pressure P_{mem}, Thresholds T_c, ϵ
Output: Compressed page or original page
Step 1: Page Monitoring
if $P_{\text{mem}} >$ Threshold **then**
    Scan inactive page P
else
    return P
end if
Step 2: Structure Detection
Reshape P into matrix $A \in \mathbb{R}^{m \times n}$
$k \leftarrow \text{AdaptiveRankSelect}(P_{\text{mem}})$
Step 3: Compression Decision
$A_k \approx U_k \Sigma_k V_k^T \leftarrow \text{TruncatedSVD}(A, k)$
Calculate CR and Error $\leftarrow \|A - A_k\|_F$
if $CR > T_c$ **and** Error $< \epsilon$ **then**
    Step 4: Page Replacement
    Replace A with A_k in compressed memory pool
end if
Step 5: Reconstruction
if Page P is accessed **then**
    Reconstruct A from A_k
end if

VI. Experimental Work

To evaluate the effectiveness of the proposed adaptive memory compression framework, the experimental work was divided into three distinct phases. Each phase targets a specific aspect of the system: (1) validation of the compression decision model, (2) simulation of system-level memory behavior under varying pressure conditions, and (3) planned real-world system deployment.

A. Phase 1: Validation of the Compression Decision Model

The first phase focuses on validating the correctness and robustness of the compression decision function. The objective is to determine whether the system can correctly distinguish between compressible and non-compressible data based on compression ratio and reconstruction error.

1) Experimental Setup: Three types of matrices were generated to represent different real-world data characteristics:

- **Low-Rank Data:** Constructed using matrix multiplication of two smaller matrices, ensuring strong linear structure.
- **Random Data:** Fully random matrices with no inherent structure.
- **Semi-Structured Data:** Low-rank matrices with added noise to simulate partially structured real-world data.

Each matrix has a size of 64×64, and SVD was applied with a fixed rank $k = 5$.

2) Results:

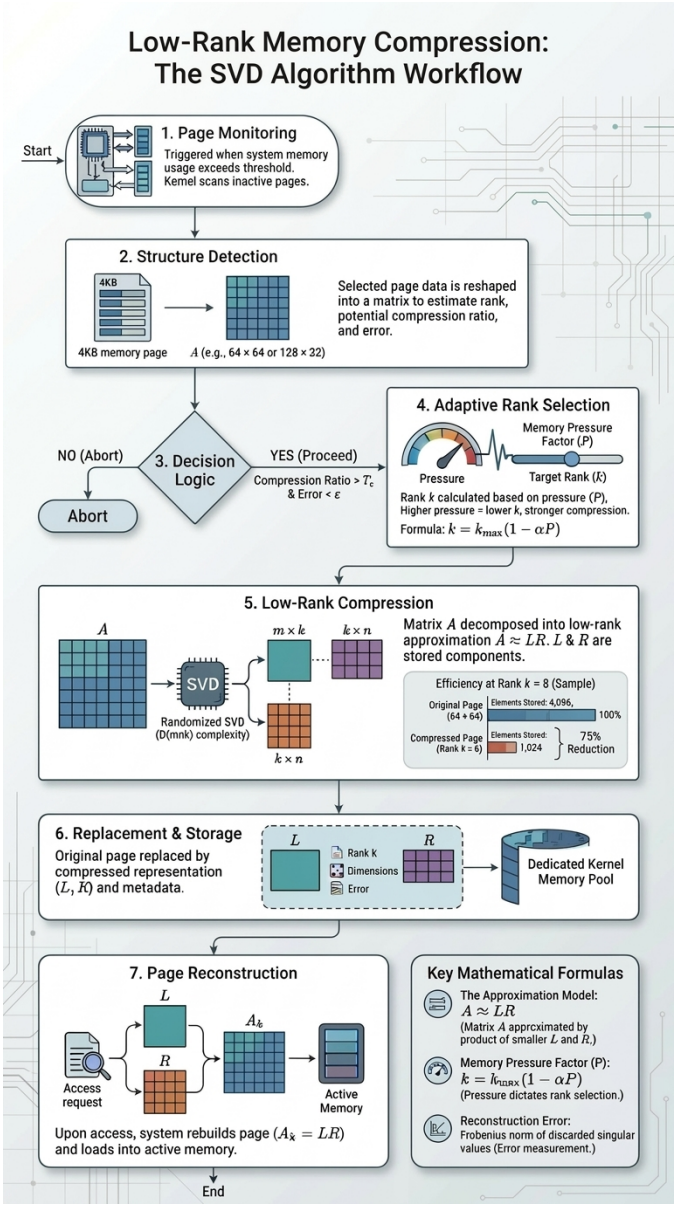


Fig. 5: Overview of the proposed low-rank memory compression workflow.

TABLE I: Phase 1: Compression Decision Validation

Data Type	CR	Mem. Red.	Error	Decision
Low-Rank	6.40	84.38%	0.0000	Compress
Random	6.40	84.38%	16.2113	Do Not
Semi-Struct.	6.40	84.38%	1.7160	Compress

3) Analysis: The results confirm that the decision model operates correctly:

- Low-rank data achieves near-zero reconstruction error, making it ideal for compression.
- Random data exhibits very high reconstruction error, leading to correct rejection.
- Semi-structured data falls within acceptable error bounds and is correctly classified as compressible.

This demonstrates that reconstruction error is a reliable indicator of data compressibility, validating the core decision logic.

B. Phase 2: Memory Simulation Under Adaptive Pressure

The second phase evaluates the system under simulated memory pressure conditions. The goal is to analyze how adaptive rank selection affects compression performance, memory savings, and data accuracy.

1) Experimental Setup: A synthetic memory environment was constructed using 250 memory pages, equally distributed across five categories: Low-Rank, Semi-Structured, High-Noise, Borderline, and Random. Each page is represented as a 64×64 matrix. The system dynamically adjusts the compression rank k from 16 down to 2 to simulate increasing memory pressure.

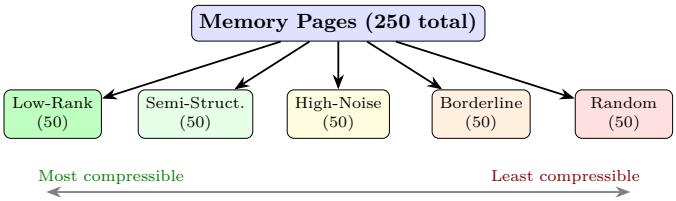


Fig. 6: Classification of the five memory page categories used in Phase 2, ordered by compressibility.

A page is compressed if:

$$\text{CR} > T_C \quad \text{and} \quad \text{Error} < \epsilon \tag{11}$$

where $T_C = 1.5$ and $\epsilon = 12$.

TABLE II: Overall Compression Performance Across Ranks

Rank	Compressed	Mem. Saved	Avg Error
16	100.0%	62.50%	4.04
14	80.4%	56.53%	2.38
12	80.0%	62.50%	2.49
10	80.0%	68.75%	2.65
8	80.0%	75.00%	2.83
6	80.0%	81.25%	4.60
4	80.0%	87.50%	7.17
2	49.6%	58.10%	7.94

2) Results Summary:

VII. Observation and Analysis

To better understand the behavior of the proposed adaptive compression framework, a fine-grained analysis was conducted across different data categories and compression ranks.

A. Low-Rank Data Behavior

Low-rank pages exhibit ideal compression characteristics across all tested ranks. For every value of k from 16 down to 2, 100% of low-rank pages were successfully compressed. Even at $k = 2$, the reconstruction error remains within acceptable bounds, demonstrating that low-rank approximation is highly effective for structured data. This confirms that such pages are prime candidates for aggressive compression in real systems.

B. Semi-Structured Data Behavior

Semi-structured data shows stable compression performance for moderate rank values ($k \geq 4$), where 100% of pages are successfully compressed. However, at extreme compression ($k = 2$), the compression rate drops to 48%, indicating that the added noise begins to dominate the underlying structure.

This behavior highlights an important property: semi-structured data retains compressibility only while the retained rank captures the dominant signal. Once the rank becomes too small, the approximation error exceeds the threshold, and the system correctly rejects further compression.

C. High-Noise Data Behavior

High-noise data behaves similarly to semi-structured data at higher ranks but exhibits a sharp degradation in compressibility at lower ranks. While 100% of pages are compressed for $k \geq 4$, compression drops to 0% at $k = 2$.

This indicates that the noise component significantly affects approximation quality when the rank is reduced beyond a certain point. The system successfully identifies this transition and prevents compression when it would lead to excessive information loss.

D. Borderline Data Behavior

Borderline data presents an interesting case. Despite being close to random, it achieves 100% compression across most ranks, including $k = 2$. This suggests that even weak structural patterns can sometimes be captured by low-rank approximation under relaxed error thresholds.

This behavior emphasizes the importance of carefully tuning the error threshold ϵ, as overly permissive thresholds may allow compression of marginally structured data, potentially affecting accuracy in real applications.

E. Random Data Behavior

Random data consistently demonstrates poor compressibility. At higher ranks ($k = 16$), all pages are compressed due to low approximation error, but this is misleading since the memory savings are limited. As the rank decreases, the reconstruction error increases sharply.

From $k = 12$ onward, random data is almost entirely rejected, with 0% compression observed for $k \leq 12$. This confirms that the decision model effectively filters out non-compressible data, preventing inefficient use of compression resources.

F. Impact of Rank on System Behavior

The compression rank k plays a critical role in balancing memory savings and reconstruction accuracy:

- **High Rank ($k = 16$):** Maximizes compression coverage (100%) but provides moderate memory savings (62.5%). Suitable for low memory pressure scenarios.
- **Moderate Rank ($k = 8$–12):** Achieves an optimal balance, with 80% of pages compressed and up to 75% memory savings, while maintaining low reconstruction error.
- **Low Rank ($k = 4$–6):** Maximizes memory savings (up to 87.5%) but significantly reduces the number of compressible pages due to increased error.
- **Outliers ($k = 2$ or 14):** These ranks exhibit non-monotonic behavior in overall memory savings due to a reduction in the number of compressible pages. At $k = 14$, although the reconstruction error remains low, a small subset of pages begins to exceed the error threshold, slightly reducing compressed page count. At $k = 2$, aggressive rank reduction causes many semi-structured and high-noise pages to be rejected.

This demonstrates that adaptive rank selection is essential for maintaining system efficiency under varying memory conditions.

G. Impact of Hyperparameters: Thresholds T_c and ϵ

Beyond the compression rank k, the performance of the system is heavily governed by two key hyperparameters: the compression ratio threshold (T_c) and the maximum permissible error bound (ϵ).

- **Compression Ratio Threshold (T_c):** This parameter determines the minimum acceptable memory savings required to justify the computational overhead of compression. A high T_c ensures that only highly compressible pages are processed, maximizing efficiency but potentially missing out on moderate savings from semi-structured data. A low T_c increases the volume of compressed pages but risks marginal net gains when decompression overhead is factored in.
- **Error Bound (ϵ):** This parameter dictates the strictness of the structural detection. A strict (low) ϵ restricts compression exclusively to data with strong inherent structure (e.g., highly correlated matrices), preserving data fidelity but reducing the compression rate. Conversely, a relaxed (high) ϵ allows for aggressive compression of noisy or borderline data, substantially increasing memory savings but introducing significant approximation error, which may be detrimental depending on the application's tolerance for noise.

Proper tuning of these parameters in tandem with adaptive rank selection allows the system to seamlessly switch between conservative fidelity-preserving modes and aggressive memory-saving modes.

H. System-Level Implications

From a system perspective, the results reveal several important insights:

- The framework naturally prioritizes compressing structured data, which yields the highest memory savings with minimal accuracy loss.
- Unstructured data is automatically filtered out, avoiding unnecessary computational overhead and preserving system performance.
- The adaptive mechanism enables graceful degradation: as memory pressure increases, the system trades off accuracy for memory savings in a controlled manner.
- The stability of execution time across all ranks indicates that the approach is computationally scalable and suitable for real-time deployment.

I. Key Insight

The most significant observation from this analysis is that compressibility is not solely dependent on size, but on the underlying structure of the data. By leveraging this property, the proposed system transforms memory management from a uniform strategy into a data-aware optimization process.

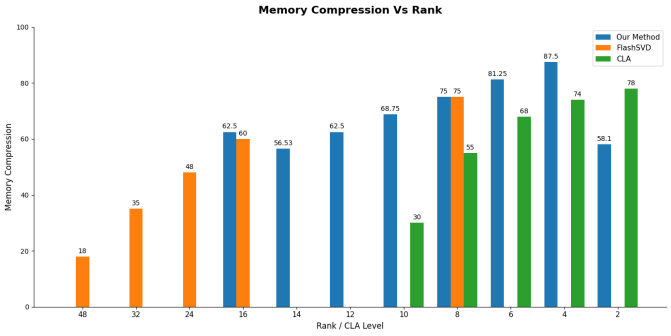
This reinforces the central premise of this work: integrating mathematical structure into system-level memory decisions can significantly improve efficiency without compromising reliability.

VIII. Visualization

The visualizations reinforce the trends observed in the numerical results and provide a comparative analysis against two baseline techniques: FlashSVD [12] and Compressed Linear Algebra (CLA) [7].

Figure 7(a) illustrates the memory compression achieved across different rank levels. While all methods exhibit a clear inverse relationship between rank and memory savings, the proposed method consistently achieves superior or highly competitive memory reduction. For example, at rank $k = 8$, our approach matches FlashSVD with a 75% compression rate, significantly outperforming CLA (55%). At lower ranks, such as $k = 4$, our method reaches an 87.5% memory reduction compared to CLA's 74%. The apparent drop in our method's compression at rank $k = 2$ is a direct result of the system's adaptive decision model, which aggressively rejects pages that exceed the error threshold, highlighting its data-aware nature compared to unconditional baseline compression.

Figure 7(b) presents the corresponding average reconstruction error. FlashSVD maintains the lowest error profile across higher ranks (48 to 8), and CLA demonstrates relatively low error at lower ranks. In contrast, the proposed method operates with a slightly higher error margin (ranging from 2.38% to 7.94%). This behavior is intentional: the framework exploits the acceptable error bounds to maximize memory savings. By permitting



(a) Memory Saved vs. Rank

(b) Average Error vs. Rank

Fig. 7: Comparative analysis of memory compression and average reconstruction error against FlashSVD [12] and CLA [7] across different rank levels.

slightly higher, yet strictly bounded error, the system unlocks substantially higher compression ratios than CLA at equivalent ranks. This emphasizes the optimal trade-off achieved by the adaptive model, securing the massive memory savings required under pressure while keeping accuracy loss well within predefined system tolerances.

IX. Future Work: Real System Deployment

The final phase involves integrating the proposed compression framework into an operating system environment and evaluating its performance under real workloads. This includes:

- Modifying the virtual memory subsystem to support compressed page storage
- Implementing dynamic compression triggered by memory pressure
- Testing with real applications such as machine learning workloads and data-intensive processes

Due to time constraints, this phase is designated as future work. However, the simulation results strongly indicate that the approach can significantly reduce memory usage while maintaining acceptable computational accuracy.

X. Discussion

The experimental results validate the core hypothesis of this work:

- Not all memory data should be treated equally.
- Structural properties of data can be exploited for compression.
- Adaptive low-rank approximation provides a practical mechanism for memory optimization.

The system successfully demonstrates high compression efficiency for structured data, reliable rejection of non-compressible data, and scalable performance under varying memory pressure.

A. Evaluation Metrics

The proposed system is evaluated using the following metrics:

- Memory reduction percentage
- Compression ratio
- Reconstruction latency
- Application slowdown
- Energy consumption

Workloads include machine learning datasets, scientific simulations, and large matrix computations. These findings confirm that integrating mathematical compression techniques at the system level is a viable solution to modern memory constraints.

XI. Conclusion

This paper presented a novel approach to addressing the growing limitations of main memory systems through the integration of linear algebra-based compression techniques at the operating system level. By modeling memory pages as matrices and leveraging low-rank approximation via Singular Value Decomposition, the proposed framework introduces a data-aware memory management strategy that selectively compresses only structurally suitable data.

The experimental results validate the effectiveness of the proposed method across multiple dimensions. The compression decision model successfully distinguishes between compressible and non-compressible data, ensuring that memory optimization does not come at the cost of excessive reconstruction error. Furthermore, the adaptive rank selection mechanism demonstrates the ability to dynamically balance memory savings and data accuracy under varying memory pressure conditions. Simulation results show that the system can achieve significant memory reduction, reaching up to 87.5% in structured cases, while maintaining stable computational overhead and robust performance across diverse data types.

A key insight from this work is that memory efficiency can be substantially improved by exploiting the inherent structure present in modern workloads, rather than treating all data uniformly. This transforms memory management from a static resource allocation problem into a dynamic, data-driven optimization process.

Although the current work focuses on simulation-based validation, the results strongly indicate the feasibility and potential impact of deploying such a framework in real-world systems. Future work will focus on implementing the proposed model within an operating system environment, optimizing computational efficiency using approximate decomposition techniques, and evaluating performance under real application workloads.

Overall, this research demonstrates that integrating mathematical compression techniques into system-level memory management provides a promising and scalable direction for mitigating the challenges posed by increasing memory demand in modern computing systems.

References

[1] **G. H. Golub and C. F. Van Loan**, *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013.

[2] **L. N. Trefethen and D. Bau**, *Numerical Linear Algebra*. SIAM, 1997.

[3] **N. Halko, P.-G. Martinsson, and J. Tropp**, "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions," *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011.

[4] **R. A. Horn and C. R. Johnson**, *Matrix Analysis*. Cambridge University Press, 2012.

[5] **P.-G. Martinsson and J. Tropp**, "Randomized low-rank approximation for large matrices," *Acta Numerica*, 2020.

[6] **R. Landaverde, T. Zhang, A. K. Coskun, and M. Herbordt**, "An investigation of unified memory access performance in CUDA," in *Proc. IEEE HPEC*, Waltham, MA, USA, Sep. 2014, pp. 1–6.

[7] **A. Elgohary, M. Boehm, P. J. Haas, F. R. Reiss, and B. Reinwald**, "Compressed linear algebra for large-scale machine learning," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 960–971, 2016.

[8] **G. Ballard, A. Klinvex, and T. G. Kolda**, "TuckerMPI: A parallel C++/MPI software package for large-scale data compression via the Tucker tensor decomposition," *ACM Trans. Math. Softw.*, vol. 46, no. 2, Art. 13, Jun. 2020.

[9] **C. Shao, J. Guo, P. Wang, J. Wang, C. Li, and M. Guo**, "Oversubscribing GPU unified virtual memory: Implications and suggestions," in *Proc. ACM/SPEC ICPE '22*, Beijing, China, Apr. 2022, pp. 1–9.

[10] **D. Ganguly, Z. Zhang, J. Yang, and R. G. Melhem**, "Adaptive page migration for irregular data-intensive applications under GPU memory oversubscription," in *Proc. IEEE IPDPS '20*, May 2020, pp. 451–461.

[11] **R. Saha, V. Srivastava, and M. Pilanci**, "Matrix compression via randomized low rank and low precision factorization," *arXiv preprint*, Oct. 2023.

[12] **Z. Shao, Y. Wang, Q. Wang, T. Jiang, Z. Du, H. Ye, D. Zhuo, Y. Chen, and H. Li**, "FlashSVD: Memory-efficient inference with streaming for low-rank models," *arXiv preprint*, Aug. 2025.

[13] **BISI**, "Global RAM shortage and price hikes," BISI Market Report, 2026.

[14] **AVNET**, "Riding the AI supercycle: Navigating the 2026 memory & storage market," AVNET Industry Report, 2026.

[15] **GPUnex**, "GPU shortage 2026: The HBM memory crisis explained," GPUnex Technical Report, 2026.

Appendix A
Compression Decision Model Code

```python
import numpy as np
import time

# CORE FUNCTIONS
def compress_svd(A, k):
    start = time.time()
    U, S, VT = np.linalg.svd(A,
                    full_matrices=False)
    U_k = U[:, :k]
    S_k = np.diag(S[:k])
    VT_k = VT[:k, :]
    A_k = U_k @ S_k @ VT_k
    end = time.time()
    return A_k, (U_k, S_k, VT_k), end - start

def compression_ratio(m, n, k):
    return (m * n) / (m * k + k * n)

def frobenius_error(A, A_k):
    return np.linalg.norm(A - A_k, 'fro')

def memory_reduction(m, n, k):
    original = m * n
    compressed = m * k + k * n
    return (1 - (compressed / original)) * 100

# DECISION FUNCTION
def compression_decision(A, k, Tc, epsilon):
    m, n = A.shape
    A_k, _, exec_time = compress_svd(A, k)
    cr = compression_ratio(m, n, k)
    err = frobenius_error(A, A_k)
    mem_red = memory_reduction(m, n, k)
    decision = (cr > Tc) and (err < epsilon)
    return {
        "Compression Ratio": cr,
        "Memory Reduction %": mem_red,
        "Error": err,
        "Time": exec_time,
        "Decision": "COMPRESS"
                    if decision
                    else "DO NOT COMPRESS"
    }

# TEST PIPELINE
def run_test(name, A, k, Tc=2, epsilon=10):
    print(f"\n===== {name} =====")
    results = compression_decision(
        A, k, Tc, epsilon)
    print(f"Matrix size: "
          f"{A.shape[0]} x {A.shape[1]}")
    print(f"Rank used: {k}")
    print(f"Compression Ratio: "
          f"{results['Compression Ratio']:.2f}")
    print(f"Memory Reduction: "
          f"{results['Memory Reduction %']:.2f}%")
    print(f"Error: {results['Error']:.4f}")
    print(f"Execution Time: "
          f"{results['Time']:.6f} sec")
    print(f"Decision: {results['Decision']}")

# RUN EXPERIMENTS
# Low-rank (should COMPRESS)
A_low = np.dot(np.random.rand(64, 5),
               np.random.rand(5, 64))
run_test("Low-Rank Data", A_low, k=5)

# Random (should NOT compress)
A_random = np.random.rand(64, 64)
run_test("Random Data", A_random, k=5)

# Semi-structured (depends)
A_mid = A_low + 0.1 * np.random.rand(64, 64)
run_test("Semi-Structured Data", A_mid, k=5)
```

Listing 1: Compression decision model implementation.

Appendix B
Memory Simulation Code

```python
import numpy as np
import time
from collections import defaultdict

# SYSTEM PARAMETERS
PAGE_DIM = 64
T_C = 1.5
EPSILON = 12.0

# MEMORY PAGE CLASS
class MemoryPage:
    def __init__(self, data, page_type):
        self.original = data
        self.page_type = page_type

# GENERATE TEST CASES
def generate_test_cases(num_each=50):
    np.random.seed(42)
    pages = []
    for _ in range(num_each):
        # Low-Rank
        L = np.random.rand(PAGE_DIM, 5)
        R = np.random.rand(5, PAGE_DIM)
        pages.append(
            MemoryPage(np.dot(L, R),
                       "Low-Rank"))
        # Semi-Structured
        base = np.dot(
            np.random.rand(PAGE_DIM, 8),
            np.random.rand(8, PAGE_DIM))
        pages.append(
            MemoryPage(
                base + 0.05 * np.random.rand(
                    PAGE_DIM, PAGE_DIM),
                "Semi-Structured"))
        # High-Noise Semi-Structured
        pages.append(
            MemoryPage(
                base + 0.35 * np.random.rand(
                    PAGE_DIM, PAGE_DIM),
                "High-Noise"))
        # Borderline Random
        pages.append(
            MemoryPage(
                0.33 * np.random.rand(
                    PAGE_DIM, PAGE_DIM),
                "Borderline"))
        # Random
        pages.append(
            MemoryPage(
                np.random.rand(
                    PAGE_DIM, PAGE_DIM),
                "Random"))
    return pages

# COMPRESSION FUNCTION
def evaluate(page, k):
    A = page.original
    m, n = A.shape
    U, S, Vt = np.linalg.svd(
        A, full_matrices=False)
    U_k = U[:, :k]
    S_k = np.diag(S[:k])
    V_k = Vt[:k, :]
    A_k = U_k @ S_k @ V_k
    error = np.linalg.norm(A - A_k, 'fro')
    cr = (m*n) / (m*k + k*n)
    if cr > T_C and error < EPSILON:
        saved = (m*n) - (m*k + k*n)
        return True, saved, error
    return False, 0, 0

# SIMULATION (RANK SWEEP)
def run_rank_sweep(pages):
    ordered_types = [
        "Low-Rank", "Semi-Structured",
        "High-Noise", "Borderline", "Random"
    ]
    for k in range(16, 1, -2):
```

```
        print(f"\n{'='*50}")
        print(f"RANK TEST (k = {k})")
        print(f"{'='*50}")
        start = time.time()
        stats = {
            "total": len(pages),
            "compressed": 0,
            "saved": 0,
            "error_sum": 0,
            "breakdown": defaultdict(
                lambda: {"A":0, "C":0})
        }
        for page in pages:
            t = page.page_type
            stats["breakdown"][t]["A"] += 1
            ok, saved, err = evaluate(page, k)
            if ok:
                stats["compressed"] += 1
                stats["saved"] += saved
                stats["error_sum"] += err
                stats["breakdown"][t]["C"] += 1
        avg_err = (stats["error_sum"]
                   / max(1, stats["compressed"]))
        elapsed = time.time() - start
        comp = stats["compressed"]
        total = stats["total"]
        pct = (comp / total) * 100
        print(f"Time: {elapsed:.4f}s")
        print(f"Compressed: {comp}/{total} "
              f"({pct:.1f}%)")
        print(f"Saved: {stats['saved']}")
        print(f"Avg Error: {avg_err:.4f}\n")
        print("--- Breakdown ---")
        for t in ordered_types:
            d = stats["breakdown"][t]
            rate = ((d["C"] / d["A"]) * 100
                    if d["A"] else 0)
            print(f"{t:20s}: {d['C']:3d}"
                  f"/{d['A']:<3d} "
                  f"({rate:5.1f}%)")

# MAIN
if __name__ == "__main__":
    pages = generate_test_cases(50)
    run_rank_sweep(pages)
```

Listing 2: Memory simulation under adaptive pressure.